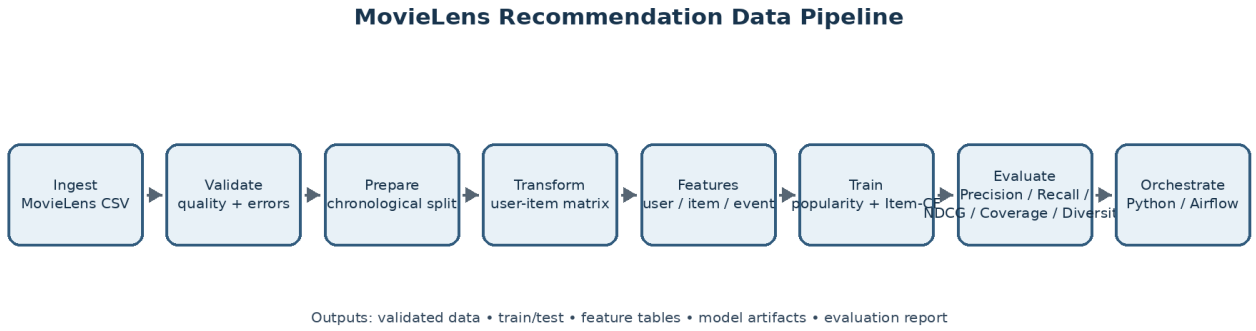
End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning — Assignment 2

1. Dataset and objective

I implemented an end-to-end movie recommendation pipeline using the official MovieLens latest-small dataset published by GroupLens Research. It contains user ratings and movie metadata; user IDs are anonymized and no demographic attributes are provided. I use the ratings as user-item interactions and the movie genres/title as item metadata. Dataset source: https://grouplens.org/datasets/movielens/. I acknowledge Harper and Konstan (2015), The MovieLens Datasets: History and Context, ACM TiiS 5(4), DOI: 10.1145/2827872.

2. Pipeline architecture and ingestion



I download and extract the GroupLens archive on first execution. I normalize ratings.csv into interactions, movies.csv into item metadata, and create a user table containing only anonymized IDs because MovieLens does not provide demographic details. Unix timestamps are converted to UTC. The validator checks required fields, duplicate user-item pairs, rating range and half-star increments, timestamp parsing and referential integrity. Invalid rows are quarantined; validation counts are shown below.

Validation summary

Input rows: 100836; valid rows: 100836; invalid rows: 0.

3. Preparation and feature management

I sort interactions chronologically and reserve the latest 20%% as a test set, keeping the earlier 80%% for training. I export user activity and item popularity summaries, a user-item rating matrix in CSV and sparse NPZ formats, and reusable user, item and interaction feature tables. User features include activity count, average rating, preferred genre and days since last interaction. Item features include popularity, average rating, title and genres. Interaction features retain rating, UTC timestamp and event type.

4. Model training and evaluation

I compare a popularity baseline, which ranks movies by training rating count, with personalized item-item collaborative filtering. The personalized model computes cosine similarity between sparse movie rating vectors and scores candidates using a user's observed ratings. I save both trained models with joblib and record experiment parameters/artifacts locally; MLflow logging is enabled when the optional dependency is installed.

The evaluation uses top-five recommendations. Precision@5 measures relevant recommendations per five slots; Recall@5 measures the fraction of held-out relevant movies recovered; NDCG@5 discounts lower-ranked hits; Coverage measures the fraction of catalog items recommended to at least one test user; Diversity is the proportion of unique genre labels in recommendation lists. RMSE and MAE are not applicable because the models rank items rather than predict numeric ratings.

Model	Precision@5	Recall@5	NDCG@5	Coverage	Diversity
Popularity	0.4690	0.0360	0.4788	0.0037	0.5636
Item-item collaborative filtering	0.0328	0.0043	0.0317	0.0084	0.5521

5. Orchestration, reproducibility and limitations

The Python workflow executes stages in dependency order from one command; an optional Airflow DAG invokes the same workflow. The split fraction and top-K setting are configurable. This dataset is for education/development and does not establish production performance. Limitations include anonymized users without demographic context, cold-start users/items, and the absence of online serving, drift monitoring and production retraining. MovieLens terms must be followed; this assignment uses the dataset for educational/non-commercial work.

Run summary: 610 users; 9742 movies; 100836 ratings; 80669 train rows; 20167 test rows.